



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

机器学习及应用实验报告

lab1——基于 KNN 的手写数字识别

王 烨

年级：2024 级

专业：计算机科学与技术

指导教师：卫金茂

2026 年 9 月 30 日

目录

一、 实验目的	1
二、 数据集预处理及划分	1
(一) 数据预处理	1
(二) 数据划分	1
1. 留一法	1
2. 10 折分层交叉验证	1
3. 标准划分与数据增强	2
三、 实验环境	2
四、 实验思路	2
(一) 实验原理	2
1. KNN	2
2. CNN	3
3. 性能评价指标	3
(二) 初级要求: 手写 KNN 分类器	4
(三) 中级要求: 手写 KNN 与 Weka 对比	5
(四) 高级要求: CNN 与数据增强	5
五、 实验结果解释及分析	6
(一) 留一法下 k 值与 ACC 的关系	6
(二) 手写 KNN 与 Weka 的对比结果	7
(三) CNN 性能评估	9
1. 标准测试	9
2. 鲁棒性测试	10
六、 实验总结	10

一、实验目的

- 掌握 KNN 分类算法的基本原理与实现方法，并采用留一法交叉验证算法的泛化能力，评估不同 k 值条件下算法对手写数字的识别精度。
- 在不同数据集下对比分析手写 KNN 与机器学习包或平台（如 weka）中的 KNN 分类器结果。
- 采用旋转等手段对原始数据进行处理，并选用 CNN 或其他深度学习方法实现手写数字识别，评估不同方法的性能差异。

本人实验的所有代码，实验结果，包括但不限于可视化和算法脚本等都可以在 github 仓库 <https://github.com/wangyeG2/Machine-learning-and-its-application> 中查看。

二、数据集预处理及划分

（一）数据预处理

起初，我们启用 semeion.data 数据集。该数据集包含 1593 个样本，每个样本为 16×16 像素的手写数字图像，标签为 0-9 的数字类别，本数据集用于验证手写 KNN 分类器的算法性能和正确性。

随后，我们从剑桥大学 AT&T 实验室官网下载了 ORL 数据集。该数据集包含 400 个样本，每个样本为 112×92 像素的面部图像，本数据集用于对比分析手写 KNN 与机器学习包或平台（如 weka）中的 KNN 分类器结果。

此外，我们对 ORL 数据集进行数据增强，增强后的数据集用于 CNN 或其他深度学习方法的性能评估。

（二）数据划分

1. 留一法

留一法是交叉验证的特殊形式，原理是将数据集划分为 N 份（ N 为样本总数），每次取 1 份作为测试集，其余 $N - 1$ 份作为训练集，重复 N 次实验，最终精度为 N 次实验的平均精度。其优点是评估结果稳定，无随机划分误差，适合小样本数据集。

我们在 semeion.data 数据集上使用留一法进行实验，评估不同 k 值条件下手写 KNN 分类器的识别精度。

2. 10 折分层交叉验证

10 折分层交叉验证，是交叉验证的一种常用方法，原理是将数据集随机划分为 10 个大小相近的互斥子集，并在划分过程中保持每一折内各类别样本的比例与原始数据集一致。每次实验取其中 1 折作为测试集，其余 9 折作为训练集，重复 10 次实验，最终精度为 10 次实验的平均精度。与留一法相比，其计算代价显著降低，且在偏差与方差之间取得了良好的平衡，适合类别分布均衡的中等规模数据集。

我们在 ORL 数据集上使用 10 折分层交叉验证进行实验，对比分析手写 KNN 与机器学习包或平台（如 weka）中的 KNN 分类器结果。

3. 标准划分与数据增强

标准划分是指将数据集按照一定比例划分为训练集和测试集。通常情况下，训练集占比为 60%-80%，测试集占比为 10%-20%。在划分过程中，应确保各类别样本的比例与原始数据集一致，以避免类别不平衡问题。

数据增强是指通过对原始数据进行一系列变换（如旋转、平移、缩放、翻转等）来生成新的样本，从而增加训练数据的多样性，提高模型的泛化能力。

我们对原始的 ORL 数据集采用标准划分方法划分为训练集和测试集，并对训练集进行旋转等数据增强手段，用于 CNN 或其他深度学习方法的性能评估。

三、实验环境

实验环境搭建如下表所示：

配置项	初级要求	中级要求	高级要求
虚拟环境	.venv	.venv	.venv-dl
Python 版本	3.13.5	3.13.5	3.13.5
pip 库	numpy	numpy、scikit-learn、 matplotlib	torch、torchvision、pillow、 scikit-learn、matplotlib
外部软件	无	Weka 3.8.6 + JDK 17	无

表 1: 实验环境

四、实验思路

(一) 实验原理

1. KNN

K 近邻 (K-Nearest Neighbors, KNN) 是属于监督学习中的一种分类算法，其基本思想是：对于一个待分类的样本，计算其与训练集中所有样本的距离，选取距离最近的 k 个样本，根据这 k 个样本的类别标签进行投票表决，最终将待分类样本归入票数最多的类别。

设训练集为 $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ ，其中 $\mathbf{x}_i \in \mathbb{R}^d$ 为样本特征向量， y_i 为类别标签。对待测样本 $\mathbf{x}$ ，kNN 首先计算其与全部训练样本的距离，本实验采用欧氏距离：

$$d(\mathbf{x}, \mathbf{x}_i) = \|\mathbf{x} - \mathbf{x}_i\|_2 = \sqrt{\sum_{j=1}^d (x_j - x_{i,j})^2} \quad (1)$$

取距离最小的 k 个训练样本构成近邻集合 $N_k(\mathbf{x})$ ，再按多数票规则输出分类结果：

$$\hat{y} = \arg \max_c \sum_{(\mathbf{x}_i, y_i) \in N_k(\mathbf{x})} \mathbb{I}(y_i = c) \quad (2)$$

其中 $\mathbb{I}(\cdot)$ 为指示函数。平票时取类别序号最小者。

在本实验中， k 值会较为显著地影响 KNN 的性能： k 过小易受噪声干扰而过拟合， k 过大则会把较远的、可能异类的样本纳入投票而欠拟合。

2. CNN

卷积神经网络 (Convolutional Neural Network, CNN) 是面向图像等网格结构数据的深度前馈网络, 其基本思想是: 通过卷积层提取局部特征, 利用池化层降低特征维度和计算复杂度, 并通过全连接层进行分类。CNN 的主要组成部分包括卷积层、激活函数、池化层和全连接层。

本实验的 CNN 由四组“卷积—批归一化—ReLU—池化”模块与一个含 Dropout 的全连接分类头级联而成, 各部分原理如下:

(1) 卷积层。利用可学习的 3×3 卷积核在特征图上滑动, 对局部感受野加权求和, 提取边缘、纹理等局部特征; 通过局部连接与权值共享大幅减少参数量。设第 l 层卷积核为 $\mathbf{K}^{(l)}$ 、偏置为 $b^{(l)}$, 输出位置 (u, v) 处的响应为

$$z_{u,v}^{(l)} = \sum_c \sum_{i=1}^3 \sum_{j=1}^3 K_{i,j,c}^{(l)} a_{u+i-1, v+j-1, c}^{(l-1)} + b^{(l)} \quad (3)$$

每个卷积层后接批归一化, 以稳定训练、加速收敛; 随后经 ReLU 激活引入非线性:

$$a_{u,v} = \max\left(0, \gamma \frac{z_{u,v} - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta\right) \quad (4)$$

(2) 池化层。前三个模块各接 2×2 最大池化下采样, 在压缩维度、降低计算量的同时增强平移不变性:

$$p_{u,v} = \max_{(i,j) \in \Omega_{u,v}} a_{i,j} \quad (5)$$

第四个模块改用自适应平均池化, 将特征图汇聚为 3×3 , 兼起全局特征整合作用。

(3) 全连接层与输出。末层特征图展平为 $256 \times 3 \times 3 = 2304$ 维向量, 经全连接层、ReLU 与 Dropout 映射为 40 维类别得分 $\mathbf{z}$, 再由 softmax 输出各类别后验概率, 取最大者为预测类别:

$$\hat{y}_c = \frac{\exp(z_c)}{\sum_{c'=1}^{40} \exp(z_{c'})}, \quad \hat{y} = \arg \max_c \hat{y}_c \quad (6)$$

网络全部参数通过最小化带标签平滑的交叉熵损失进行学习:

$$L = - \sum_{c=1}^{40} \tilde{y}_c \log \hat{y}_c \quad (7)$$

其中 $\tilde{y}_c$ 为平滑后的目标分布。训练采用 Adam 优化器, 配合权重衰减与余弦退火学习率, 梯度由反向传播算法计算并迭代更新。

在本实验中, 网络结构与训练配置会较为显著地影响 CNN 的性能: 层数与通道数过少则特征提取能力不足而欠拟合, 过多则在小样本条件下参数冗余、易过拟合; 训练轮数过少网络未充分收敛, 过多则会记忆训练噪声; 学习率过大易震荡甚至发散, 过小则收敛缓慢。本实验在轻量网络基础上配合 Dropout、权重衰减与标签平滑等正则化手段, 并通过四方向旋转扩充训练集, 以提升模型对旋转测试样本的鲁棒性。

3. 性能评价指标

- **精度 (Accuracy, ACC):** 分类正确的样本数占总样本数的比例:

$$\text{ACC} = \frac{\text{正确分类样本数}}{\text{总样本数}} \quad (8)$$

- **归一化互信息 (Normalized Mutual Information, NMI):** 衡量预测标签与真实标签的相似度, 取值范围 $[0, 1]$, 值越大表示一致性越高。本实验采用算术平均归一化:

$$\text{NMI}(Y, \hat{Y}) = \frac{I(Y, \hat{Y})}{(H(Y) + H(\hat{Y}))/2} \quad (9)$$

其中:

- $I(Y, \hat{Y})$ 为互信息, $I(Y, \hat{Y}) = H(Y) - H(Y | \hat{Y})$;
- $H(Y)$ 为真实标签的熵, $H(Y) = -\sum_{i=1}^C P(y_i) \log_2 P(y_i)$;
- $H(\hat{Y})$ 为预测标签的熵, $H(\hat{Y}) = -\sum_{j=1}^C P(\hat{y}_j) \log_2 P(\hat{y}_j)$;
- C 为类别数;

- **混淆熵 (Confusion Entropy, CEN)**: 基于混淆矩阵衡量分类结果的不确定性, 值越小表示分类越明确。设混淆矩阵为 $\{n_{jk}\}$, n_{jk} 为真实类别为 j 而被预测为类别 k 的样本数, M 为总样本数, 令

$$D_j = \sum_{k=1}^C n_{jk} + \sum_{k=1}^C n_{kj}, \quad p_{jk} = \frac{n_{jk}}{D_j}, \quad w_j = \frac{D_j}{2M} \quad (10)$$

其中 D_j 为与类别 j 相关的样本总数, w_j 为对应权重, 并约定 $0 \log 0 = 0$ 。CEN 定义为

$$\text{CEN} = -\sum_{j=1}^C w_j \sum_{\substack{k=1 \\ k \neq j}}^C \left(p_{jk} \log_{2(C-1)} p_{jk} + p_{kj} \log_{2(C-1)} p_{kj} \right) \quad (11)$$

即对每个类别 j , 将其双向错分概率 (预测错与被错判) 以 $2(C-1)$ 为底求熵并按 w_j 加权求和。

(二) 初级要求: 手写 KNN 分类器

我们编写了 knn.py, 这份代码加载了数据集 semeion.data, 利用 numpy 库和数学展开对朴素二重循环的 KNN 进行了向量优化, 并在留一法交叉验证下评估了不同 k 值条件下的识别精度。核心代码如下:

Listing 1: knn.py

```

1 def pairwise_sqdist(X): #计算样本间的平方欧氏距离
2     sq = np.einsum("ij,ij->i", X, X)
3     d2 = sq[:, None] + sq[None, :] - 2.0 * (X @ X.T)
4     np.maximum(d2, 0.0, out=d2)
5     return d2
6
7 def majority_vote(neighbor_idx, y, k, n_classes): # 多数投票
8     n = neighbor_idx.shape[0]
9     labels = y[neighbor_idx[:, :k]]
10    flat = labels + np.arange(n)[:, None] * n_classes
11    counts = np.bincount(flat.ravel(), minlength=n * n_classes).reshape(n,
12                                n_classes)
13    return counts.argmax(axis=1).astype(np.int64) # 平票取最小标签
14
15 def knn_loo(X, y, k_list=K_LIST, n_classes=N_CLASSES): # 留一法KNN
16     neighbor_idx = topk_neighbor_idx(pairwise_sqdist(X), max(k_list))
17     acc = {}
18     for k in k_list:
19         preds = majority_vote(neighbor_idx, y, k, n_classes)

```

```

19     acc[k] = float((preds == y).mean())
20     return acc

```

(三) 中级要求：手写 KNN 与 Weka 对比

我们从剑桥大学 AT&T 实验室官网拉取了 ORL 数据集¹，并下载了 weka 3.8.6，利用 explorer 进行实验。由于 weka 的 KNN 分类器 (IBk) 默认使用 10 折分层交叉验证，我们改造了 knn.py，实现手写 KNN 分类器与 weka 中的 KNN 分类器在相同基准下的对比实验。

此外，我们追加了 NMI 和 CEN 两个评价指标，这一指标计算同时对手写 KNN 分类器和从 weka 导出的测试结果进行。核心代码如下：

Listing 2: knn_weka.py

```

1 def knn_cv(X, y, k_list=K_LIST, n_classes=N_CLASSES, n_splits=N_SPLITS, seed=
   SEED):
2     skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
3     cm = {k: np.zeros((n_classes, n_classes), np.int64) for k in k_list}
4     pred_all = {k: np.empty(len(y), np.int64) for k in k_list}
5     for tr, te in skf.split(X, y):
6         nb = np.argsort(pairwise_sqdist_ab(X[te], X[tr]), axis=1, kind="
           stable")[:, :max(k_list)]
7         for k in k_list:
8             pred = majority_vote(nb, y[tr], k, n_classes)
9             pred_all[k][te] = pred
10            for t, p in zip(y[te], pred):
11                cm[k][t, p] += 1
12    acc = {k: float(np.trace(m) / m.sum()) for k, m in cm.items()}
13    return acc, cm, pred_all

```

此外，我们还引入了 sklearn 库的 KNN 分类器作为对照，验证手写 KNN 分类器的正确性，此处不再赘述。

(四) 高级要求：CNN 与数据增强

我们编写了 cnn.py。在程序中，我们将原始 ORL 数据集以 1:1 的比例划分为训练集和测试集，对原始训练集进行 CNN 训练，随后对训练集执行四个方向的旋转增强，对规模扩大 4 倍后的训练集再进行 CNN 训练。

训练采用 Adam 优化器配合余弦退火学习率调度，损失函数为带标签平滑的交叉熵。训练结束后，在原始测试集下进行测试，并与 KNN 进行对照。

此外，为验证数据增强带来的鲁棒性收益，我们将测试集旋转后重新送入各模型，并与原始测试集上精度最高的 k 值所对应的 KNN 作对照。核心代码如下：

Listing 3: cnn.py

```

1 # CNN训练过程
2 def train_cnn(model, loader, epochs, lr, tag): # Adam+余弦退火+标签平滑

```

¹下载地址：<https://www.cl.cam.ac.uk/research/dtg/attarchive/facedatabase.html>，对于 windows 系统选择.zip 文件下载解压，解压结果为 att_faces 文件夹，下属 s1-s40 文件夹，各文件夹内为 10 张.pgm 文件，代表 40 个不同人在 10 个不同角度的共计 400 张面部图像。

```

3     model.train()
4     opt = torch.optim.Adam(model.parameters(), lr=lr,
5                             weight_decay=WEIGHT_DECAY)
6     sch = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
7     crit = nn.CrossEntropyLoss(label_smoothing=LABEL_SMOOTH)
8     for ep in range(1, epochs + 1):
9         for xb, yb in loader:
10             xb, yb = xb.to(DEVICE), yb.to(DEVICE)
11             opt.zero_grad()
12             loss = crit(model(xb), yb)
13             loss.backward(); opt.step()
14         sch.step()
15     return model
16
17 @torch.no_grad()
18 def cnn_predict(model, X, mean, std, batch=256): # 批量推理
19     model.eval()
20     dl = DataLoader(FaceDataset(X, np.zeros(len(X)), mean, std),
21                     batch_size=batch, shuffle=False)
22     return np.concatenate(
23         [model(xb.to(DEVICE)).argmax(1).cpu().numpy() for xb, _ in dl])
24
25 # 旋转测试集, 进行鲁棒性测试
26 for a in [0] + list(args.angles):
27     Xt = X_te if a == 0 else rotate_images(X_te, a)
28     ao = evaluate_preds(y_te, cnn_predict(cnn_orig, Xt, mean, std))[0]
29     aa = evaluate_preds(y_te, cnn_predict(cnn_aug, Xt, mean, std))[0]
30     nb = np.argsort(pairwise_sqdist_ab(Xt, X_tr),
31                     axis=1, kind="stable")[:, :best_k]
32     ak = float((majority_vote(nb, y_tr, best_k) == y_te).mean())
33     print(f"{name:<10} CNN-orig={ao:.4f} CNN-aug={aa:.4f} "
34           f"kNN(k={best_k})={ak:.4f}")

```

五、实验结果解释及分析

(一) 留一法下 k 值与 ACC 的关系

knn.py 在 semeion.data 的测试结果如下:

k	ACC
5	0.9040
7	0.9102
9	0.9109
11	0.9090
13	0.9046

表 2: 不同 k 值对应的识别精度

由上表可知，在留一法交叉验证下， $k = 9$ 时手写 KNN 分类器的识别精度最高，为 91.09%，随着 k 值增加，ACC 呈现先上升后下降的趋势，说明在本数据集下中等规模的邻居数能够在过拟合与欠拟合之间取得较好的 trade-off，与前文所述结论一致。

(二) 手写 KNN 与 Weka 的对比结果

我们将 knn-weka.py 的运行结果导出为 orl_results.csv，并与 Weka 的运行结果共同整理为表格，结果如下：

表 3: ORL 数据集上不同 K 值的测试结果对比

	k	knn.py	Weka	Sklearn
ACC	5	0.9100	0.9025	0.9100
	7	0.8875	0.8750	0.8875
	9	0.8375	0.8275	0.8375
	11	0.8025	0.8000	0.8025
	13	0.7675	0.7675	0.7675
NMI	5	0.9323	0.9244	0.9323
	7	0.9214	0.9152	0.9214
	9	0.8919	0.8913	0.8919
	11	0.8698	0.8818	0.8698
	13	0.8530	0.8565	0.8530
CEN	5	0.0515	0.0576	0.0515
	7	0.0593	0.0642	0.0593
	9	0.0823	0.0837	0.0823
	11	0.0997	0.0927	0.0997
	13	0.1130	0.1134	0.1130

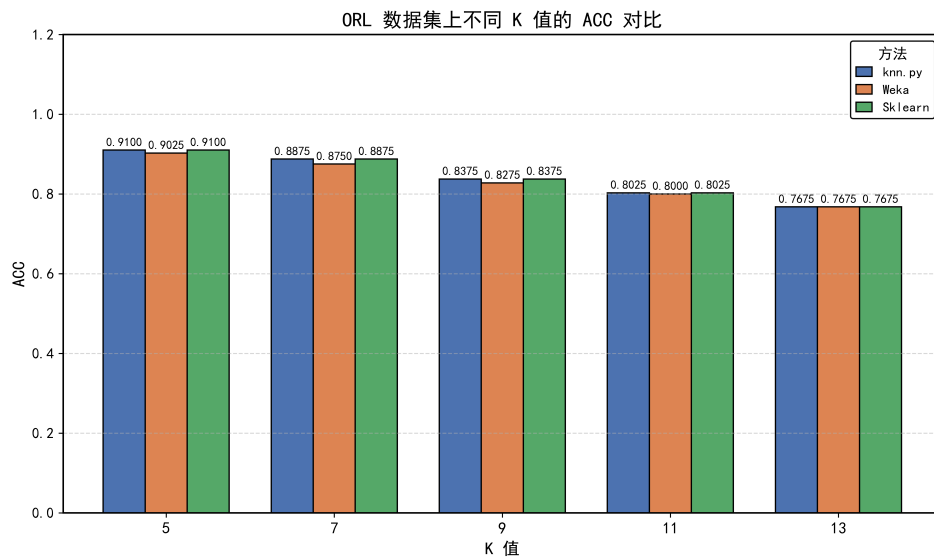
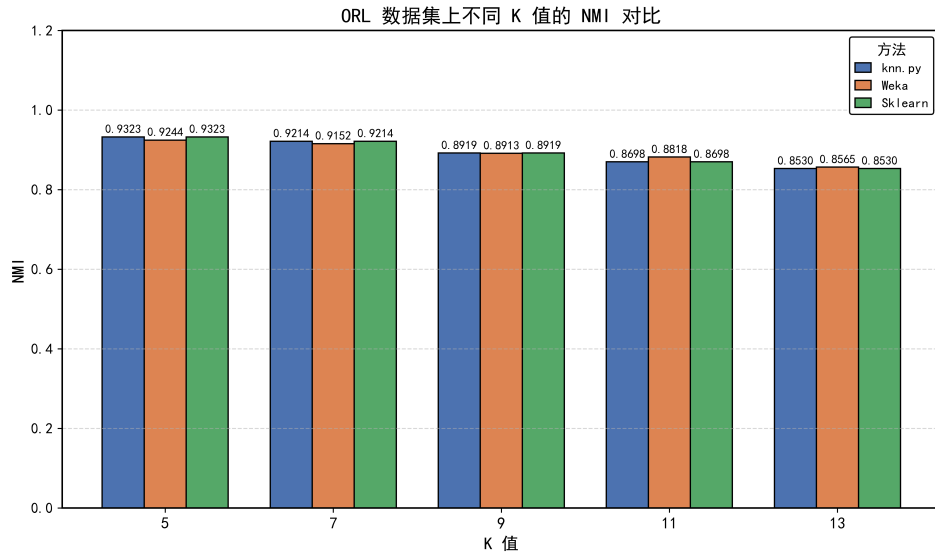
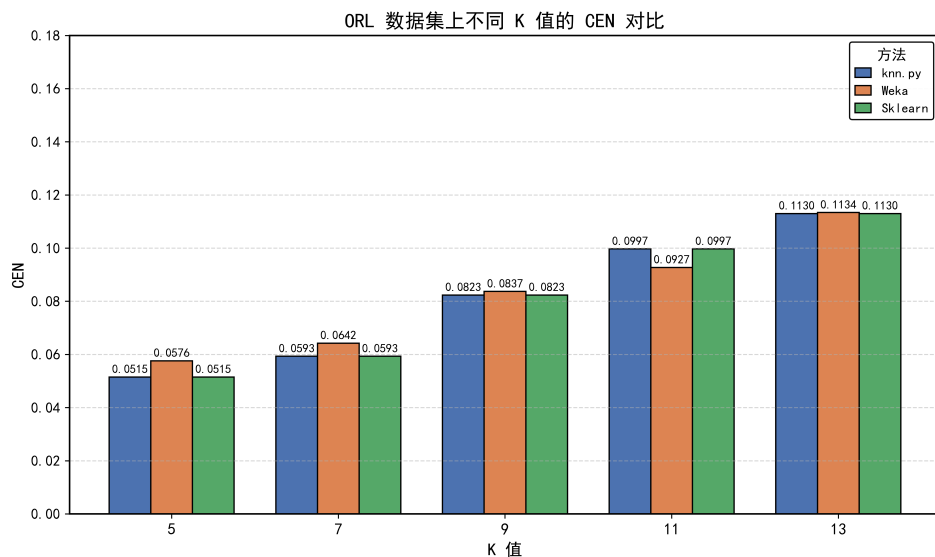


图 1: ORL 数据集上不同 K 值的 ACC 对比

图 2: ORL 数据集上不同 K 值的 NMI 对比图 3: ORL 数据集上不同 K 值的 CEN 对比

分析上述数据，可得出如下结论：

- 手写 KNN 与 scikit-learn 实现等价：由表可知，knn.py 与 Sklearn 两列在所有 K 值下的 ACC、NMI、CEN 均完全相同，原因在于二者的实验条件受控一致：共用 10 折分层交叉验证划分、采用欧氏距离、平票处理规则一致。
- 手写 KNN 与 Weka 的测试结果高度一致：Weka 与自实现结果的 ACC 差距极小，换算为样本数即在 400 个测试样本中仅差 1 ~ 5 个，NMI 与 CEN 的差距同样在 0.01 以内，且三者随 K 变化的趋势完全一致。
- K 值在本数据集下的影响显著：随着 K 从 5 增大到 13，ACC 和 NMI 均显著地单调递减，CEN 显著地单调递增。其原因在于 ORL 数据集每人仅有 10 张图像， K 过大时多数投票会将距离较远、可能属于其他人的样本卷入决策，造成欠拟合。

- 部分数据的反常：整体而言，ACC 与 NMI 同向变化、与 CEN 反向变化，符合“分类越准则归一化互信息越大、混淆熵越小”的理论关系。但以 $K = 11$ 时为例，Weka 的 ACC 低于自实现，其 NMI 却更高、CEN 更低。这是因为 NMI 与 CEN 度量的是错误在类别上的分布结构，而非错误的绝对数量。

对于手写 KNN 与 Weka 的差异分析，可能原因如下：

- 交叉验证划分不同：Weka Explorer 默认 10 折交叉验证的随机化方式与手写 KNN 的随机数生成器实现不同，导致折划分不一致、测试集不同，少数边界样本的预测发生翻转。
- 数值量化精度损失：knn.py 中 to_arff 以保留 4 位小数的方式写入特征，Weka 读入的是舍入后的数据，欧氏距离出现微小扰动，在近邻边界处可能引起次序互换。
- 平票与近邻排序规则不同：IBk 多数类平票时倾向偏向距离更近的邻居，而自实现取较小类别的标签；Weka 的 LinearNNSearch 与 numpy 稳定排序对等距样本的次序处理也不同。

（三）CNN 性能评估

我们将 cnn.py 的运行结果导出为 cnn.log，提取数据并整理后，结果如下：

1. 标准测试

表 4: 原始测试集 (0°) 上各方法性能对比

方法	k	ACC	NMI	CEN
CNN-orig	—	0.9850 ± 0.0087	0.9895 ± 0.0061	0.0079 ± 0.0045
CNN-aug (增强 5×)	—	0.9683 ± 0.0076	0.9845 ± 0.0052	0.0113 ± 0.0042
KNN-orig	5	0.7500 ± 0.0000	0.8665 ± 0.0000	0.1057 ± 0.0000
	7	0.7050 ± 0.0000	0.8401 ± 0.0000	0.1223 ± 0.0000
	9	0.6500 ± 0.0000	0.8204 ± 0.0000	0.1349 ± 0.0000
	11	0.6400 ± 0.0000	0.8097 ± 0.0000	0.1401 ± 0.0000
	13	0.6100 ± 0.0000	0.7987 ± 0.0000	0.1534 ± 0.0000
KNN-aug	5	0.7500 ± 0.0000	0.8665 ± 0.0000	0.1057 ± 0.0000
	7	0.7050 ± 0.0000	0.8401 ± 0.0000	0.1223 ± 0.0000
	9	0.6500 ± 0.0000	0.8204 ± 0.0000	0.1349 ± 0.0000
	11	0.6400 ± 0.0000	0.8097 ± 0.0000	0.1401 ± 0.0000
	13	0.6100 ± 0.0000	0.7987 ± 0.0000	0.1534 ± 0.0000

注：加粗为该列最优值

实验结果符合预期：在相同的训练集下，CNN 在测试集的表现远超 KNN，原因在于 CNN 通过卷积层级式地学到“边缘 → 局部器官 → 人脸身份”的判别性特征，而 KNN 直接在原始像素上算欧氏距离，对光照、微小位移、姿态都极其敏感，这导致了面对复杂数据时 KNN 的性能出现明显瓶颈。

此外, CNN-aug 相比于 CNN-orig 在原始数据集下的识别精度略有下降, 这是因为 CNN-aug 在容量更大, 特征更多的数据集下需要兼顾各方面的识别效果, 对于原始正立方向的拟合出现一定牺牲, 本实验中为了保证 CNN-aug 发挥出最佳效果, 我们增加了训练轮数直至模型收敛。而牺牲一定精度换取的鲁棒性提升是非常巨大的。

2. 鲁棒性测试

表 5: 旋转测试集上的鲁棒性对比 (指标: ACC)

测试集	CNN-orig	CNN-aug (增强 5 $\times$)	KNN ($k=5$)
原始 (0°)	0.9850 $\pm$ 0.0087	0.9683 $\pm$ 0.0076	0.7500 $\pm$ 0.0000
左上 (45°)	0.1133 $\pm$ 0.0535	0.9567 $\pm$ 0.0161	0.1150 $\pm$ 0.0000
左下 (135°)	0.0500 $\pm$ 0.0050	0.9550 $\pm$ 0.0132	0.0550 $\pm$ 0.0000
右下 (225°)	0.0350 $\pm$ 0.0229	0.9567 $\pm$ 0.0076	0.0450 $\pm$ 0.0000
右上 (315°)	0.1283 $\pm$ 0.0058	0.9583 $\pm$ 0.0029	0.0550 $\pm$ 0.0000

注: KNN 取原始测试集上最优的 $k = 5$; 加粗为该行最优值。

实验结果完全佐证了以下结论:

- CNN 不具备旋转不变性: 这是由 CNN 的算法原理决定的, 卷积和池化层的数学原理导致模型本身只具备平移不变性, 即对于数据的线性变化, CNN 可以保持训练精度稳定, 但是一旦引入旋转因子, 旋转后的图像数据本质是分布外输入, 与模型学习到的特征是无交集的, 这导致了 CNN-orig 的测试结果崩溃, 模型性能退化为随机识别水平。
- 数据增强手段可以提升鲁棒性: 通过旋转等数据增强手段, 扩大训练集规模和特征, 可以使 CNN 强制学习新的特征, 以此提升模型的泛化能力, 在新的测试集下仍能表现出较好的性能。

六、实验总结

本次实验通过手动实现 KNN 分类器, 并与 Weka、sklearn 等机器学习包和平台的 KNN 分类器进行性能比较, 并引入 CNN 等深度学习手段, 在 semeion 手写数字与 ORL 人脸两个数据集上完成了多层次、系统性的算法实践与对比分析, 充分说明了 k 值对于 KNN 的显著影响, CNN 相较于 KNN 的显著性能优势, 数据增强手段在提升模型泛化能力方面的显著效果。